Andon.Cloud Release Safety Plan

A. Core rule: define what “safe” even means

You need 3 classes of guarantees:

A1) Hard invariants (ship-blockers)

If any fail → **do not ship**, regardless of stats:

- **No data loss**: node/edge counts preserved (except explicit migrations), no orphaning beyond known rules
- **Schema integrity**: all constraints satisfied (types, required properties, unique IDs)
- **Referential integrity**: every edge references existing nodes; no dangling pointers
- **Deterministic replay**: same event log → same graph state (within tolerance if timestamps/ordering exist)
- **Idempotency**: reapplying migration or replay does not multiply entities

A2) Semantic invariants (almost-hard)

If these shift too much → **ship-block**:

- **Anchor query stability** (see Probe Suite below)
- **Key relationship preservation** for designated “anchor entities”
- **No new high-impact cycles** (unless expected): eg sudden emergence of tight loops in traversal

A3) Statistical drift (tolerances)

This is where your 2σ / 3σ thinking lives — but applied correctly.

B. Probe Suite (the “graph traversal tests” you always run)

Pick ~100–500 fixed probes across representative customers or synthetic workloads. Keep them stable.

B1) Anchor seeds

Maintain a curated set of:

- high-degree nodes
- high-centrality nodes
- random sample nodes
- “golden” domain anchors (if any)

B2) Query probes (run pre/post upgrade)

For each seed set:

1. **Top-K neighbors** (K=10/25/50) by your ranking rule
2. **k-hop reachability size** (k=1..4)
3. **Shortest-path distances** to a fixed target set
4. **Subgraph induced by filters** (counts + density)
5. **Centrality metrics** snapshot deltas (pagerank/degree/betweenness if feasible)
6. **Edge activation frequency** under a replayed workload (if you have event logs)

Output each probe as a numeric vector so you can do drift math.

C. Statistical thresholds (practical + defendable)

C1) Per-metric z-score gates (recommended)

For each metric family, compute baseline mean/σ from repeated runs on the **control** instance (same version, multiple runs) to estimate "noise."

Then for candidate version:

- **Beta pass** if:
    - **0 ship-block metrics > 3σ**, AND
    - **≤ 1% of metrics exceed 2σ**, AND
    - **no single anchor probe changes rank-order > X** (see C2)
- **Ship (GA) pass** if:
    - **0 metrics > 3σ**, AND
    - **≤ 0.1% of metrics exceed 2σ**, AND
    - drift is stable across at least **N=3** independent runs

Why: one metric spiking is different from broad drift. This captures both.

C2) Rank stability gates (crucial for "meaning")

For Top-K neighbor lists:

- Compute **Jaccard similarity** between pre/post neighbor sets.
    - **Beta**: median Jaccard ≥ **0.85**, 5th percentile ≥ **0.70**
    - **GA**: median ≥ **0.92**, 5th percentile ≥ **0.80**
- For ordering, use **Kendall tau** or **Spearman**:

- **Beta**: median ≥ **0.75**
- **GA**: median ≥ **0.85**

If your product depends heavily on “who is connected to what,” these are more meaningful than raw σ.

C3) Distribution shift gates (when σ lies)

Graph metrics are often heavy-tailed. Add one distribution test per family:

- **Wasserstein distance** or **KS test** against control distribution
  - **Beta**: no family shows “large” shift (define a threshold per family from historical “safe” updates)
  - **GA**: all families within historical safe bands

If you want a dead-simple substitute: track **p50/p95/p99** for each metric family and gate on percent change:

- **Beta**: p95 and p99 change ≤ **10%**
- **GA**: p95 and p99 change ≤ **5%**
  (Adjust if your graphs are inherently volatile.)

D. Multi-instance test matrix (your 10-instance plan, formalized)

Run 10 instances as follows:

D1) Roles

- **1 Control**: frozen version, frozen schema; only receives mirrored inputs (read-only if possible)

- **3 Canary Upgrade**: upgrade path under normal workload replay
- **3 Fault Upgrade**: upgrade path with injected faults
- **3 Reversion Upgrade**: upgrade then revert testing (see section F)

D2) Input model

Use the same workload replay across all instances:

- same events
- same ordering
- same timing (or deterministic timing)
- same random seeds

Otherwise your stats get noisy.

E. Fault-injected divergence testing (must-have)

You want faults that mimic real customer pain:

E1) Fault types

- **Partial migration**: crash mid-migration, restart, continue
- **Network partition**: DB write delays, split brain prevention checks
- **Out-of-order events**: reorder a slice of ingestion events
- **Duplicate events**: idempotency check
- **Corrupted payloads**: malformed nodes/edges, missing fields

- **Resource starvation**: disk full, memory pressure, CPU throttling
- **Rollback during write**: trigger revert while writes happen (this is the nightmare scenario)

E2) Fault success criteria

- **No invariant violations** (A1)
- **Bounded blast radius**: corruption (if any) is detectable and quarantined
- **Recovery convergence**: after fault removal, probe drift returns within thresholds
- **Time-to-recover** within defined SLO (you define it)

F. Reversion testing (the truth: “revert” isn’t safe unless you design for it)

You’re right: reverting after writes on the new version *will* affect the graph unless you control writes/migrations.

There are only 3 safe revert models:

Model F1) Snapshot Revert (simplest, strongest)

Rule: upgrade requires a **frozen backup** (snapshot) taken at upgrade start.

- Upgrade proceeds
- If revert → restore snapshot (full state rollback)
- **No attempt** to “run old code on new state”

Testing checklist

- Verify snapshot restore produces identical probe outputs as pre-upgrade baseline
- Verify event replay from snapshot point matches control

This is the cleanest “mandate frozen backups” posture.

Model F2) Dual-write / Blue-Green DB (what you described)

Two DBs:

- DB-A runs old version and remains source of truth
- DB-B is upgraded and receives mirrored writes (or replays)
- Only flip traffic to DB-B when stability confirmed

Key rule: the “new” system must be able to ingest the same write stream without semantic divergence beyond thresholds.

Testing checklist

- Compare DB-A and DB-B probe suite continuously during mirroring
- Gate cutover on drift thresholds
- Validate cutover + cutback (flip back) does not lose events

This is robust but heavier operationally.

Model F3) Event-log as source-of-truth (best long-term)

If your ingestion is already event-based:

- Keep an append-only event log
- DB is a derived materialization
- Upgrade = rebuild materialization on new version from event log

- Revert = continue using old materialization

This is the most principled approach and makes “revert” real.

Testing checklist

- Deterministic rebuild validation
- Replay correctness under faults
- Performance + cost

G. Release gates (simple “go/no-go” table)

G1) Beta gate

- ✅ All hard invariants pass (A1)
- ✅ Rank stability: median Jaccard ≥ 0.85, p5 ≥ 0.70
- ✅ 0 metrics > 3σ and ≤ 1% > 2σ
- ✅ Fault suite: no unrecoverable divergence; recovery returns within Beta drift bands
- ✅ Revert model validated (F1 snapshot restore OR F2 blue-green OR F3 rebuild)

G2) GA gate

- ✅ All Beta gates plus:
- ✅ Rank stability: median Jaccard ≥ 0.92, p5 ≥ 0.80
- ✅ ≤ 0.1% metrics > 2σ
- ✅ 72-hour soak under workload replay with zero invariant violations

- ✅ Documented upgrade playbook + automated preflight checks

H. Operational mandates for customers (what you enforce)

This is where you reduce liability and chaos:

1. **No forced hot updates**: pinned versions only
2. **Upgrade requires one of**:
    - Snapshot backup (mandatory)
    - Blue-green DB
    - Event-log rebuild path
3. **Writes during upgrade**:
    - either paused (“maintenance window”), or
    - mirrored via blue-green, or
    - logged to event log for deterministic replay
4. **One-click export/restore** tooling (customer trust + safety)